



Transcript

Week 23: Generative AI- Part 2

Video 1 - Advanced Generative Models

Have you ever wondered how an A-I system can write a poem, generate realistic images, or even compose music? Generative A-I is not just about analysing data—it's about creating something new from it. You'll uncover how these systems work and why they're shaping the future of creativity and innovation.

Let's take a closer look at Advanced Generative Models and why they matter so much in today's AI world.

At the heart of these models is their ability to produce **high-quality data**. That means they can generate data that looks almost **identical** to what we'd find in real life—just by learning from a **training dataset**. Think of it like **teaching an AI** to paint a photo-realistic picture after showing it hundreds of real photos.

But these models don't stop there. They go beyond basic or vanilla models by tackling harder challenges—like improving **output quality**, generating a **wide range of data**, and dealing with **more complex data types**. This includes avoiding common issues **such as mode collapse**, where a model keeps repeating the same kind of result.

And what's more, advanced models offer enhanced capabilities. Using **smarter architectures** and training methods, they can create data that's not **just more accurate**—but also more diverse **and more specific**. For example, they can generate content that matches certain criteria, like creating **medical images** tailored to a specific condition or realistic dialogue for a particular scenario.

In short, advanced generative models are not just tools for creating content—they're pushing the boundaries of what AI can do, opening up exciting new possibilities in research, industry, and creative applications.

Let's now explore some of the most advanced generative modelling techniques in artificial intelligence today. These approaches **build on the foundations** of Generative Adversarial Networks, or GANs, and Variational Autoencoders, known as V-A-Es. Over time, researchers have developed powerful new methods to **generate realistic and diverse data**. We'll focus on four of the most **influential**: Conditional GANs, Style GAN, Transformers for Text Generation, and Diffusion Models.

First, we have Conditional GANs, often called C-GANs. These models allow us to condition the generated output on specific input information, **such as labels or reference** images. That means we can tell the model **what kind of content** we want it to generate. For example, if we provide **the digit "seven" as a label**, a C-G-A-N can produce an image of a handwritten seven. This technique gives us more control and makes the generation process more purposeful.

Next is Style GAN, one of the most influential advances in image generation. It **creates extremely realistic and** high-quality images, especially faces. What makes StyleGAN unique is its ability to **control fine-grained details**—like facial expressions, lighting, and even age. For example, you can generate **different versions of the same person**: smiling, frowning, young, or old—simply by adjusting style parameters. It offers an impressive level of realism and control.

Moving beyond images, we come to Transformers for Text Generation. These models, like G-P-T-3, short for Generative Pre-trained Transformer 3, use a mechanism called **self-**



attention to understand and generate human-like language. They can write coherent paragraphs, **translate languages**, **answer** questions, and even power dialogue systems like chatbots. Their strength lies in **generating text that** is not only fluent, but also contextually relevant and meaningful.

Lastly, we have Diffusion Models. These are a newer but extremely powerful class of generative models. They begin **with random noise and gradually** refine it into high-quality data—such as images, audio, or video—by reversing a noising process step by step. This gradual transformation **leads to stunning results in realism** and diversity. They've **made headlines for** their performance in tools like DALL-E 2 and Imagen.

Each of these techniques pushes the boundary of what generative AI can achieve—across images, text, and sound—bringing us closer to creating synthetic content that is nearly indistinguishable from the real world.

Now let's dive into Conditional Generative Adversarial Networks, also known as C-GANs.

These models take the basic idea behind **traditional GANs**—where a Generator creates data and a Discriminator evaluates it—and add a powerful twist: they include **extra input**. That input could be a class label, a reference image, or any other type of **data that helps guide** the output.

So instead of generating random content, Conditional GANs allow us to steer the process. For example, if you give the model a **label like “seven,”** it won't just make any digit—it will generate an image that clearly looks like the number seven.

This added control means we can generate content that is more meaningful, accurate, and relevant to specific goals. It opens up many practical applications—from generating labelled training data to customising outputs for creative or scientific tasks.

Let's now explore the architecture of Conditional Generative Adversarial Networks, or C-GANs.

At first glance, the structure looks **very similar** to a standard GAN. But there's one key difference—**conditioning variables**. These give us more control over what the model creates.

A C-GAN still has **two main parts: a Generator** and a Discriminator. But both now receive an **extra input alongside** the usual random noise or data. Let's look at what that means.

The Generator takes in **two inputs**—random noise and a conditioning variable. That conditioning variable **could be a label** like “seven,” an attribute like “smiling,” or even a reference image. The Generator uses this to **create an output** that doesn't just look real—but also matches the given condition. So, if the condition **is the label “seven,”** the Generator will try to produce a clear image of the digit seven.

The Discriminator **also receives this condition**. It compares the input image and the label together. Its job is not just to decide whether the image is real or fake, but to check whether it **matches the condition**. For example, if the image **shows a “seven” and** the condition is also “seven,” it should mark it as real. But if the **condition is “two”** and the image shows a “seven,” it should flag it as fake.

By feeding the condition into both the Generator and the Discriminator, a C-GAN creates outputs that are not only realistic, but also **purpose-driven and specific**—making them far more useful for real-world applications.

Conditional GANs accept an extra input, so we can steer what the model produces instead



of sampling at random. In practice this control supports four common uses. First, **super resolution** raises the resolution of a **low-quality image** while keeping the original context and adding believable detail. Second, **semantic image segmentation** generates a **labelled map** that identifies and categorises regions in a scene, such as roads, buildings and trees, which helps with analysis and decision making. Third, **text to image generation** turns a natural language prompt **into a picture** that matches the description. Fourth, **image synthesis with attributes** lets us specify features such as **the colour of an object**, the style of a landscape or the facial features of a generated person. Together these uses show the flexibility and precision of conditional generation, making synthetic content more practical and closer to the real world.

Now, let's highlight the key advantages of Conditional GANs, or C-GANs.

First, c-GANs offer **significant versatility**. Their ability to include various conditioning inputs means they're not limited to generating random data. They can adapt to different tasks and data types, such as images and text, simply by changing the conditioning variable.

Second, they enable **controlled generation**. Unlike regular GANs, c-GANs let us directly influence the output based on specific attributes or categories. This allows us to generate specific types of data on demand, which is especially useful for practical applications that need tailored content.

Lastly, c-GANs help **improve the quality of** generated data. By using explicit conditions, the model focuses on creating relevant and high-fidelity outputs that match the given criteria. This often results in more coherent and appealing results.

Together, these advantages make c-GANs a key part of advanced generative modelling.

Let's now focus on StyleGAN, a powerful generative model known for its unique features in image synthesis.

What makes StyleGAN stand out is its **style-based architecture**. This design **separates high-level** features—like **pose or identity**—from low-level details—like colour, texture, or lighting. That means the model gives us more control over how images are created. You can adjust major aspects of a face, **such as age** or expression, without affecting **finer details** like hair texture.

StyleGAN also uses something called **progressive training**. This means the model **starts by generating** small, low-resolution images **and gradually increases** the resolution as training continues. This process improves image quality and **helps stabilise training**, allowing StyleGAN to **generate high-resolution** results that look realistic and detailed.

By combining style-based control with progressive learning, StyleGAN has become a benchmark in generating lifelike and customizable synthetic images—especially of human faces.

Let's now look at the architecture of StyleGAN, which is designed to improve image quality and give us fine control over what the model generates.

It all starts with a **latent vector input**. This is a random noise vector—just like in other GANs—but here, it doesn't go straight into the Generator.

Instead, it first passes through a **mapping network transformation**. This is a small neural network that **converts the input** into an intermediate representation, often called **W**. This step helps separate different image features, making them easier to understand and control.

Next, StyleGAN uses AdaIN style adjustment, short for Adaptive Instance Normalization.



The intermediate features from the **W space are injected** into the Generator at multiple layers. This lets us control **different details at different** stages—like pose and shape early on, and colour or texture later.

Finally, StyleGAN uses a progressive growing strategy. It **starts by generating** low-resolution images and **slowly adds more** layers to increase the size and detail of the image. This gradual training helps produce **stable, high-resolution** results.

Altogether, the mapping network, AdaIN adjustments, and progressive growth make StyleGAN a powerful model for **creating high-quality, realistic images** with impressive control.

Let's explore some of the real-world applications of **Style-GANs**, building on the powerful architecture we've already seen. These models are being used across industries that rely heavily on visuals—enabling creativity, efficiency, and access to synthetic data at an entirely new level.

We'll start with Art and Design. Style GANs can be used to create completely **new and visually** striking pieces of art. These artworks often include intricate detail and unique styles, **opening up exciting** opportunities for digital artists. Beyond entire artworks, Style GANs can also generate specific **design elements**—like textures, patterns, or visual motifs—that artists can use in larger creative projects. This can **significantly speed up** design workflows.

In **Fashion**, Style GANs help designers create new clothing designs and patterns. They can even be used to generate full virtual outfits, making it easier to explore ideas quickly. They also support style and pattern visualisation—so designers can see how a certain fabric or pattern will look on different types of garments or body types, without needing to produce physical samples.

Moving to **Medical Imaging**, Style GANs are valuable in two key ways. First, they can create synthetic medical images—which is extremely helpful when real data is limited, sensitive, or difficult to collect. Second, these synthetic images can be used in training and research. For example, they can augment datasets for medical AI models, helping to improve accuracy and robustness. Researchers can also use them to study a wide range of medical conditions without needing access to actual patient data.

Finally, in **Gaming and Animation**, Style GANs are used to generate realistic character models. These models are diverse and detailed, and they reduce the need for manual character design. They can also be used to build rich and immersive environments, generating natural-looking landscapes, buildings, or other assets that populate virtual worlds.

Each of these applications shows how Style GANs are reshaping industries by providing control, realism, and creativity—all through synthetic image generation.

Let's look at the major challenges in training Generative Adversarial Networks—especially mode collapse and training stability. **Mode collapse** happens when the generator keeps producing a small set of outputs that are good enough to fool the discriminator instead of a broad, **diverse set**. **The GAN gets** stuck on a few sample types, ignores other possibilities in the data, reduces diversity, and becomes **less useful for tasks** that need varied results.

Training can be unstable because the generator and discriminator compete; one can overpower the other or learning may fail to converge. Two fixes are common. **Wasserstein GAN** uses a different loss function that provides more informative gradients. **Spectral normalisation** controls the discriminator's strength by limiting the Lipschitz constant of each layer, preventing it from learning too quickly. These **techniques help**



stabilise training and reduce the risk of mode collapse.

To judge performance, we use evaluation metrics. **Fréchet Inception Distance (F-I-D)** measures the distance between features of real and generated images—lower scores mean higher quality and greater diversity. **The Inception Score (I-S)** measures how clearly and confidently a pre-trained Inception model classifies generated images—higher scores usually indicate better visual quality. **Understanding these** challenges—and how to address them—is essential for training and applying GANs in real-world scenarios.

If generative models can already write stories, paint images, and even design new data—what groundbreaking creation will *you* bring to life with them? Think about it.

Video 2 – Transformers for Text Generation

Ever wondered how machines can write like humans, weaving together words that feel natural and insightful?

Transformers have transformed how machines generate text by making it more human-like and contextually rich.

They use something called **an autoregressive model**—which means they predict one word at a time, based on the words that came before. A well-known **example is G-P-T-3**, which creates fluent and coherent text that often feels like it was written by a person.

These models are excellent at **understanding context**, which means they don't just string words together—they generate meaningful responses that align with the topic. As a result, they can be used across a wide **range of applications, including creative** writing, summarisation, and machine translation.

Thanks to their design, transformers **also deliver state-of-the-art** performance in many natural language processing tasks, often outperforming older methods in both accuracy and fluency.

Let's begin with an introduction to Transformers, a **deep learning architecture** that has reshaped the field of Natural Language Processing, or NLP.

Transformers marked a **major shift from older** models like Recurrent Neural Networks, known as R-N-Ns, and Long Short-Term Memory networks, or L-S-T-Ms. Those **earlier models struggled** to understand long-range relationships in text—especially when words were far apart in a sentence or paragraph.

The **key breakthrough** of Transformers is their use of self-attention mechanisms. These mechanisms allow the model to **focus on all the words** in an input sentence—regardless of their order or position. This means the model can understand the **relationship between words** even if they are separated by a large distance. That's what gives Transformers their power to generate contextually rich and coherent language.

This architecture was **first introduced** in a paper titled "*Attention Is All You Need*", published by Vaswani et al. in 2017. That paper laid the **groundwork** for many of the language models we use today—from translation systems to tools like GPT.

Thanks to this innovation, Transformers have become one of the **most important developments** in artificial intelligence—transforming how machines understand and generate human language.

Let's now explore the key components of Transformers—the building blocks that make this



architecture so powerful, especially for sequence-to-sequence tasks in natural language processing.

First, we have the Encoder–Decoder Architecture. This structure is at the heart of many sequence models. The **encoder** reads and processes the input sequence—such as a sentence—and generates a rich, context-aware representation of it. Then, the **decoder** takes that processed information and generates the output sequence one word at a time. **Think** of the encoder as summarising the meaning, and the decoder as turning that meaning into a response.

Next, the most revolutionary feature—Self-Attention. In this mechanism, every **word in the sequence** pays attention to every other word, regardless of how far apart they are. This helps the model capture relationships between words and understand the sentence as a whole. To do this, self-attention relies on three key components: the **Query, written as Q, the Key, written as K, and the Value, written as V**. These are different mathematical transformations of the input words. The model **compares** the query of one word to the keys of all other words to decide how much attention to give to each one. **Then it combines** the values accordingly.

Finally, we have Multi-Head Attention. This extends self-attention by **running it multiple times** in parallel, each time focusing on different relationships. For example, one attention head might learn grammar, another might focus on meaning. These multiple heads **are then merged** to give a more complete understanding of the input.

Together, these three components—Encoder–Decoder, Self-Attention, and Multi-Head Attention—are what give Transformers their incredible power to understand and generate language with depth and precision.

Let's now look at how Transformers work by breaking their process into three key stages: input representation, attention mechanism, and layer stacking.

First, we begin with Input Representation. Every word or token in a sentence **is converted into** a vector—a list of numbers—through a process called embedding. But because the model doesn't naturally understand the order of words, **it adds something** called positional encoding. This helps the model know the **position of each** word in the sentence, which is crucial for understanding meaning.

Next is the core of the Transformer: the Attention Mechanism. More specifically, it uses what's called **Scaled Dot-Product Attention**. The model **compares** each word to every other word using three key vectors: Query (Q), Key (K), and Value (V). These vectors are mathematical representations that help the model decide how much **focus**—or attention—each word deserves based on its context. The result is that the model **learns** which words to prioritise when generating output.

Finally, we have Layer Stacking. Transformers are made up of **multiple layers of attention** and feed-forward networks. The **output from one** layer becomes the input for the next, allowing the model to build up knowledge in stages. This helps the Transformer **learn more complex** relationships as it goes deeper—refining its understanding at each level.

This combination of input encoding, attention, and layer stacking is what **gives Transformers their** ability to process complex language with exceptional accuracy and flexibility.

Let's explore the key differences between Autoregressive and Non-Autoregressive models, especially in how they handle sequence generation.

Autoregressive models **generate one word**—or one element—at a time. Crucially, each



new word **is predicted based** on all the previously generated words in the sequence. This means the model **takes the full context into** account, aiming to predict the next most likely word. By conditioning each output on the prior ones, the model ensures **strong coherence** and contextual accuracy. This step-by-step generation makes them ideal for tasks like **text generation, where the flow** and meaning of the sentence really matter.

In contrast, Non-Autoregressive models generate the **entire sequence in parallel** or nearly all at once. Instead of building the sequence word by word, they predict **missing words or elements** using the surrounding context. This allows for **much faster generation speeds**. However, because these models don't rely on previously generated outputs, their text often **lacks the same level** of contextual flow. That makes them less suitable for tasks like natural text generation.

Instead, non-autoregressive models are more useful for tasks like **text classification or sentence-level prediction**, where understanding the full input sequence is more important than generating new flowing text.

So, while autoregressive models prioritise coherence by using prior context, non-autoregressive models prioritise speed and efficiency. Choosing the right model depends on the specific needs of your task—whether that's generating realistic, flowing language or processing sequences quickly for classification or prediction.

Now, let's shift our focus to Transformers for Text Generation, a major leap in natural language processing. These **models are pretrained** on massive text datasets and can generate responses, summaries, translations, and more, based on just a small input.

GPT-3, short for Generative Pretrained Transformer 3, is an example of an autoregressive model. It generates one word at a time, conditioning on previous words to produce fluent, human-like text. This makes it especially useful for answering questions, writing stories, or generating code.

T5, or Text-to-Text Transfer Transformer, takes a slightly different approach. It treats every task—whether translation, summarisation, or question answering—as a text generation problem. This gives it great flexibility across many language tasks.

BERT, or Bidirectional Encoder Representations from Transformers, is mainly used for language understanding. But when adapted with techniques like masked language modelling or fine-tuning, it can also contribute to text generation, particularly in classification or sentence prediction tasks.

Together, these models show how powerful Transformer-based architectures can be for generating coherent, context-aware language at scale.

Now, let's look at how Transformers are used in real-world text generation tasks. These models have changed the way we create and interact with language.

First, we have **text completion and continuation**. This is what powers autocomplete tools, where the model predicts the next word or sentence based on what you've typed. It's widely used in smart keyboards, coding assistants, and email drafting.

Next is **creative writing**. Transformer models like GPT-3 can write poems, stories, and even generate entire articles. They produce text that feels natural and flows well, often with surprising originality.

We also see Transformers in **dialogue systems**. They help build chatbots and virtual assistants that can understand context and respond meaningfully. This makes conversations feel more natural and is especially useful in customer service or personal assistants.



And finally, **machine translation**. Transformers are behind today's best translation tools. They can accurately translate text from one language to another, helping people communicate across the world.

These applications show just how powerful Transformers are for generating human-like text in so many different ways.

Now, let's explore how we can make pretrained Transformers even more useful by fine-tuning them for specific tasks. It helps the model adapt to new domains or tasks beyond its original training.

We can think of this process in three stages—starting at the top with **Zero-Shot Learning**. In this case, models like GPT-3 can respond to brand-new prompts, even without extra training on the task. That's because they've already been exposed to a massive amount of text during pretraining. For example, you can ask it to summarise an article or write an email—and it will attempt the task, even if it was never fine-tuned for that exact goal.

Next, we move **to Task-Specific Fine-Tuning**. This is when we take a pretrained model and adjust it for a focused task—like sentiment analysis or summarisation. Say we want it to detect emotions in movie reviews. We fine-tune it using labelled examples, so it learns what positive, negative, or neutral tone looks like. The model becomes much more accurate because it's trained directly on that kind of data.

Finally, at the base, we have **Transfer Learning**. This is the broadest and most powerful idea. It means we take a model pretrained on general internet text, and then fine-tune it on something specialised—like medical research papers. The model retains its deep understanding of language but now gains domain-specific knowledge. This helps it handle highly specialised tasks without needing to start from scratch.

In short, fine-tuning lets us unlock the **full potential** of large pretrained Transformers, making them more accurate, more relevant, and much more useful for real-world applications.

Now let's look at how we can improve the quality and precision of text generation from Transformer models. While these models are powerful, there are still ways to fine-tune their behaviour to get better, more controlled results.

The first technique is called Prompt Engineering. This simply means writing your prompts more carefully to get the type of output you want. Think of it like giving **clearer instructions**. The way you word your prompt really matters—it can make the difference between a vague response and a highly relevant one. A powerful strategy **here is few-shot learning**. Instead of just giving the model a task, you show it a few examples in the prompt itself. That **helps it generalise** better and handle new tasks—even without additional training.

The second technique is Controlling Output Generation. This is about managing how random or focused the model's responses are.

One way to do this is by adjusting the **temperature**. A higher temperature makes the output more creative and unpredictable, while a lower temperature gives more straightforward, focused answers.

Another option is **Top-K Sampling**. This limits the model to choosing from the top K most likely next words. By ignoring the unlikely ones, the output becomes more accurate and less "hallucinated."

There's also **Top-P Sampling**, also called nucleus sampling. Here, the model looks at the most likely words, but only keeps adding words until the combined probability reaches a



set threshold. It's a flexible way to balance creativity and control.

Together, these advanced techniques help developers and learners shape AI output more precisely—whether you want creative storytelling, accurate answers, or structured summaries.

Now let's talk about some of the challenges we face when using Transformer models for text generation. These models are powerful, but they're not perfect—and understanding their limitations is just as important as knowing their strengths.

The first challenge is called **long-range dependencies**. Transformers are better than older models like recurrent neural networks at handling long sequences, but they can still lose track of context if the input or output becomes too long. For example, when generating an entire article or story, the model may start well but gradually lose coherence as it continues.

The second issue is **biases in generated text**. Transformers are trained on large volumes of text from the internet, and that data often includes harmful stereotypes, misinformation, or biased viewpoints. Unfortunately, the model can learn and repeat those biases unless we apply careful filtering and mitigation techniques.

The third challenge is **computational cost**. Training these models takes a lot of processing power and memory. That makes them expensive to run and often inaccessible to smaller organisations or independent researchers who don't have access to high-performance hardware.

Finally, there's the problem of **hallucination**. This means the model generates text that sounds fluent and convincing but is actually wrong or misleading. It's especially risky in tasks like answering questions, where accuracy matters most. The model might give you an answer with full confidence—even if it's completely made up.

Researchers are actively working to solve these challenges. By understanding the limits of Transformers, we can make them more reliable, more fair, and more useful in real-world applications.

Let's wrap up our discussion on Transformers by looking at some real-world examples that show how these models are already transforming industries.

First, take **GPT-3 in business applications**. Generative Pre-trained Transformer 3, or GPT-3, is widely used to automate tasks like content creation, email drafting, and even writing marketing copy or social media posts. It can suggest quick responses, generate text that sounds natural, and even translate plain language into working code—boosting productivity for developers.

Next, we have **chatbots in customer service**. Many companies now rely on Transformer-based models to power their support systems. These chatbots can understand customer queries more accurately, recommend products, and solve problems without needing a human agent. That means faster response times and better customer satisfaction.

Finally, there's **story generation**. Models like GPT-3 can write full stories, novels, or articles with surprising fluency. In fact, the content they produce is often so coherent and human-like that it's hard to tell it was written by AI.

Together, these examples show just how impactful Transformer models have become. They're **not just theoretical**—they're being used right now to simplify tasks, solve problems, and spark new kinds of creativity across domains.

Now, let's look ahead to the future of transformers in text generation, which is full of



exciting possibilities.

One major development is the rise of **multimodal models**. These are models like CLIP and DALL-E, which don't just process text—they combine text and images to generate rich, detailed outputs. For example, DALL-E can turn a simple sentence like *"a futuristic cityscape at sunset"* into a highly realistic image. This makes multimodal models incredibly useful in creative fields like advertising, digital art, and game design.

Another big **trend is scaling transformers**. Researchers are working on making these models bigger, faster, and more efficient—so they can handle more complex tasks while using less computing power. Some of these new models can summarise entire books, translate between languages, or even write code—quickly and accurately. This helps bring powerful AI tools into **everyday use**, including chatbots, virtual assistants, and content creation software.

But with this growing power comes ethical responsibility. As transformers are used to generate more and more content, **concerns about misinformation**, harmful bias, or inappropriate outputs become more serious. Imagine a model that accidentally generates false news or offensive stereotypes. To prevent that, developers are focused on adding safeguards, increasing transparency, and creating clear guidelines for responsible use.

So in short, transformers are evolving rapidly—combining different types of input, **becoming more powerful** and more useful. And as they do, it's crucial to think not just about what they can do, but also how to use them ethically and wisely.

Let's talk about how researchers improve the quality and realism of the data generated by GANs, or Generative Adversarial Networks.

One powerful method is called **Progressive Growing**. Imagine a GAN that starts off by generating tiny images—just four by four pixels. Over time, the model gradually increases the resolution to eight by eight, sixteen by sixteen, and eventually reaches full detail, like one thousand twenty-four by one thousand twenty-four pixels. This smooth growth helps the model stabilize in the early stages and produce sharper, more realistic results in the end.

Another technique is **Feature Matching**. This ensures that what the GAN creates closely resembles real-world examples. For instance, when generating faces, the model learns to match overall features like the shape of the eyes, nose, and mouth—rather than trying to copy images pixel by pixel. This approach produces results that feel more natural and believable.

Finally, we have **Latent Space Regularization**. This method stops the model from simply memorizing the training data. Instead, it helps the GAN learn general patterns or structure from the data—like learning the rules behind how the data is formed. This improves the model's ability to create diverse and realistic outputs, whether it's for images, sounds, or text.

Together, these three methods push GANs to generate higher-quality, more lifelike content—helping us move closer to truly believable synthetic media.

Let's explore how advanced generative models are trained and fine-tuned using large datasets.

We start with **Scaling Up**. This means using extremely large datasets—millions or even billions of samples—and training on powerful GPUs or TPUs. For example, GPT-3 was trained on hundreds of billions of words. To handle this scale, the model needed **massive compute clusters**, which processed the data efficiently and helped it learn complex patterns.



Next is **Transfer Learning**. Instead of training a model from scratch, we use a pre-trained model that has already learned general features from large datasets. We then fine-tune it on a smaller, specific dataset. Take legal text generation, for instance. A language model like GPT can be fine-tuned on legal **documents to perform better** in that domain. This saves both time and computational resources.

Finally, we have **Data Augmentation**. This involves creating new training examples by slightly altering existing ones. In image generation, that could mean flipping, cropping, or changing colours. For text or medical images, we can generate new **synthetic examples** using generative models. This is especially useful when real data is limited—like in rare disease diagnosis, where new medical images can be created to train better AI systems.

Together, these three approaches—scaling up, transfer learning, and data augmentation—help build powerful generative models that can handle complex, real-world data.

Generative models are changing the game across industries by making new forms of content and data synthesis possible.

In **creative industries**, these models can generate digital art, compose music, or build realistic three-dimensional models for films and video games. For example, AI can create detailed cityscapes or animated characters, saving time and reducing the need for manual design work.

In **healthcare**, generative models help produce synthetic medical data that protects patient privacy. This synthetic data is then used to train diagnostic AI systems. It also helps reduce bias in datasets, leading to fairer, more accurate models.

In **fashion and design**, AI can generate new clothing prototypes or accessories based on specific styles or trends. Designers can test and explore creative ideas faster, improving product development.

And in **finance**, these models are used to detect fraud. AI generates realistic examples of suspicious transactions to help banks train anomaly detection systems. This improves their ability to catch unusual or fraudulent activity.

These applications show how generative AI is not just a research topic—it's becoming a real-world tool for solving problems and unlocking new possibilities.

Video 3: Generative AI- Part 2- Demo

Hello everyone. Welcome all. In this particular section, we're going to talk about the main idea behind transformers, which is your multi-head attention mechanism.

Traditional sequential models like RNNs and part of it is LSTMs have been powerful tools for handling sequence data, but they come with some serious limitations. The first challenge is long-term dependencies. When processing long sequences, RNNs often struggle to retain context from earlier parts of the input.

This means that important information from the beginning can get lost by the time model reaches at the end. Also, we have selective focus. Human beings naturally focus only on the most relevant part of what we read or hear.

The most relevant part of it we focus, we pay extra attention. Traditional RNNs process everything step by step, but attention mechanisms allow models to selectively focus on just the key part of the input, ignoring less important details. This makes them much more efficient and accurate for tasks like translation or summarization when we talk more attention mechanism.



And this is another challenge that we have in traditional model. Attention mechanism can provide selective focus, which your traditional model is not able to do it. So, so far we are talking about long-term dependencies as one of the limitation of the traditional model which is solved by the attention mechanism.

Another issue is selective focus, then parallelisation. There is an issue of parallelisation. RNNs are inherently sequential.

They process one step after another, which slows down training. Transformers, on the other hand, replace recurrence with attention. This shift allows computations to happen in parallel, making training much faster and more scalable.

At the heart of the attention mechanism is a very simple but powerful idea. Every token in a sequence is transformed into three vectors, query, key and value. Now, every token when I'm saying is basically every word in a sentence.

So, if I say a sentence as I love AI, basically it will be divided into individual words, which we'll call it as tokens. And then for each token, the attention mechanism creates these vectors, query, key and value vectors. First, the query vector.

Think of this as a, as the question, what am I looking for? So, it represents the current token search criteria, the lens through which it will scan the sequence to find relevant context. So, for a particular word, it says what it is looking for, what the word is looking for. Next comes the key vector.

This is like a catalogue of what information is available. Each token in the sequence provides a key answering the question, what information do I have? And finally, we have the value vector. Once the query matches with the key, the value provides the actual content we want to use.

It answers the question, what is the actual information to bring forward? Another way of looking at it would be that there is a room full of experts sitting in that particular room. And they sit with a name tag, which talks, which tells what is the name of the expert and what is their area of expertise. And then a person is entering to find out information about say black hole in the universe.

So, this person is looking for black hole in the universe. What are they? How do they, how are they formed? Then the key when we are talking about, every expert has this key, which is nothing but a name tag, which tells what is their name and what do they offer. So, probably expert 1 talks about sports and expert 2 talks about science.

So, that is what the information they have. So, what information do I have? These are the information the experts have about sports, about science. And then this person who was looking to know about black hole realised it has a good match with expert 2, who talks about science.

And then he goes to this expert 2. And then V here is the actual value that this expert is offering about black hole. He explains what black holes are, how they are formed and how many of them are there probably, if at all they can tell. So, similarly in our sentences that we are processing, a word is carrying some information.

It is looking for a key. So, e_1 , e_2 are other words in the sentence. This person who entered is say first word in the sentence.

It is looking for some kind of information. It checks the key, which is what the other words are offering. Wherever the match happens, it goes over there and it fetches the actual value that word is offering.



That is what the idea behind these three vectors are. That is what the idea about attention mechanism is. So, in essence, it works by comparing the query with the keys.

The better the match, the more weight is assigned to the corresponding value. In other words, the system decides which part of the input are most relevant and then extracts only that information for further processing. And this is how attention allows model like transformers to focus selectively, capture context effectively and handle even very long sequences without losing track of important details.

Now, to understand further, when we talk about the core idea, let us say I am saying I have a word, sorry, a sentence, I love NLP. Now, for each of these words that I have, say I will create a query vector with set of elements in it, a key vector and a value vector, which in a very simple manner, it is telling me what I is looking for, what information does it carry and what is the actual information that it has. Then for love also will have a query vector created, a key vector created and a value vector and then even for NLP will have these vectors created.

Now, remember at the start of it, these are simply initialised weights. These are nothing but weights that we need to train through our neural network training. So, these are some initial weights which are assigned, which keeps getting, which gets updated through back propagation algorithm.

So, initially we assign some value to it. Then as we go forward, we update them so that the real contextual meaning is preserved. So, this is how it will initialise.

Then here is some small example. Let's say we are talking about I love NLP and we are defining the query key and value vectors. After projections, we get this, the query for I is 1 0. So, we have taken a two-dimensional vector to explain this concept.

So, the query vector for I is this and the key vector for I is this and the value vector for I is this. So, we have taken a vector of length 2 to explain the query key and value vector and then we have done this for all three words. Okay.

So, if you look at closely over here, we have for love Q, K and V and then also for NLP. So, this is the initialised value. And then what's happening over here, we are focussing on the word, say love.

We want to know how much focus should we go give to this particular word love. So, we take the query vector of love which is 0 1 and then we start comparing with all the keys because we want to know which other word has the information that this word love is looking for. And how do we find that? We can simply find dot product of two vectors, whichever value is high that has a higher similarity.

So, here is a dot product of the query vector of love and the key vector of I. We do the dot product, get the value here 1. Then we do it with love and love because we need to calculate the self-attention also. So, here we are calculating again the value came as 1 and then what's the dot product of the word love and NLP. So, love query and the K, Q is your Q vector and the K vector, key vector, we multiply and we got 0. So, for the word love, we got similarity scores as 1 1 0. That's overall similarity score because we have three words.

So, we are writing it in a form of vector 1 1 0. So, it has a high match with the word I. It has a high match with love and it has a very low match with NLP. It's just an example. So, just take it in a hypothetical situation.

This is what we got. Now, next what we will do is we'll process this through a softmax



activation. So, what we will do over here is we'll take this value 1 1 and 0 and then we'll apply softmax activation on this.

So, this will become e to the power 1. This will become e to the power 1. This will become e to the power 0 and then sum this up. Whatever the value comes, the individual exponential score is divided by that particular sum, and that's what we got over here. 0.42, 0.42 and 0.16. So, that's my softmax score which tells us how important the word is and it has been.

Actually, using softmax, we have made sure all the attention scores are between 0 to 1 and they sum up to 1 for better understanding. So, we got 42, 0.42 here, 0.42 here, 0.16 in terms of percentage 42, 42 and 16 percent. That's the attention weights that we are calculating.

After that, once the attention weights are calculated, then we take those individual attention weights and multiply with the value vector of the each word. So, basically, we are saying that the importance of word I in the context of the word love is this much. Multiply the weight that we have got, the attention score with the value of I, value vector of I. So, we're going to multiply 0.42 with the value vector of I which is 1 comma 2 which gives me this.

Then do it for love which gives this and do it for NLP which gives us this. And then what you do is add them up along the dimensions, means this will get added over here and then over here. So, these three values will be added and then the second dimension that will be added and we'll get a value here as 1.74 and 2.74. That's what we got over here as the weighted sum of the value vector.

And the final result is this. This is the contextualised embedding of the word love. This is how the word love is represented in our sequence.

So, when it is carrying forward in our neural network, it will be represented by this vector. Based on that, eventually some kind of output will be produced. And if that output does not make sense, it is not matching the actual value, we'll calculate the loss.

Loss will travel back, will update all the weights involved in the key vector, value vector and your query vector. That's the first one. And once it gets updated, this value will change based on which the final output will be changed.

That's what the idea of working with the query key and value vector is. I should write in the same sequence. So, query key and value vector is.

That's what the whole idea is. So, it's basically the, if I tell you in a very simpler manner, in a Lehman language, the idea is this, that we are starting, we are starting with a particular given sentence and we compare, we start with the query key and value vector of each token, I would say token, for each token we'll create this. We'll compare Q and K, which will take the dot product of the query and the, on the key of words in a sequence.

This will give the similarity score that tells us how relevant each word is to the current one. So, for a particular word, let's say I, I'll find the dot product of these two, those two vectors between the query vector of I and the key vector of I, then key vector of love and then key vector of NLP, which will tell us the similarity score, which explains us how relevant each word is to the current one, which is I. Then turn the scores into weights, apply softmax, so the, so those scores become probabilities, attention weights. Higher weights means more important, the word for this context.

And then use V to extract the content, multiply each value of V vector by its attention score and sum them up. This will create a new contextualised representation of the word and rich with the information from the most relevant word. So, the idea is that we were trying to find out in a given sentence, which word is more important to a particular word



and that information we are embedding in the form of this vector, which is your, based on the attention mechanism.

So, this is how it works. That's what the working of the attention mechanism is. Now, self-attention, I explained to you entirely in detail how it works in terms of the calculation.

The process begins with computing similarity score for each token. We take its query vector and calculate dot product with the key vectors of all tokens in the sequence. This gives us a measure of how much one token should pay attention to another.

Next, we apply softmax normalisation. The raw similarity scores are converted into probabilities called attention weights. These weights tell us the relative importance of each token in the sequence with respect to the current token.

Once we have these weights, we move to weighted representation step. Here, we compute a weighted sum of the value vectors. Tokens with higher attention weights contribute more strongly, while less relevant tokens contribute very little.

This entire process is summarised in the mathematical formulation that you see there. You have $\text{attention} = \frac{\text{softmax}(\frac{QK^T}{\sqrt{d_k}})}{d_k}V$, which basically represents QK^T transpose and then multiply with the vector V and d_k is the dimension of the keys, which is a scaling factor, which prevents extremely small gradients. Vanishing gradient problem will not happen.

That's why we use it. Now, let's walk through a simple example of how self-attention works in practise. Take the sentence, the cat sat on the mat.

When the model processes the word sat, the first step is to form its query vector. This query is then compared with all the key vectors of the words in the sentence. The cat sat on the mat for each word.

Through this comparison, similarity scores are calculated. Naturally, sat will find stronger connections with words like cat and mat because they provide meaningful context. For instance, cat tells us who is doing the action and mat tells us where the action is happening.

Next, the attention mechanism takes these similarity scores and uses them as weights to combine the value vectors of all words. Since cat and mat score higher, their value vectors contribute more to the weighted sum, while less relevant words like the or on contribute less. The end result is contextualised representation of the word sat.

Instead of just being an isolated word, it now carries information about both the subject cat and the location mat. This is the power of self-attention. It allows each word to dynamically gather a context from the rest of the sentence, creating richer and more meaningful representation.

Now, we're going to talk about multi-head attention, which is a very important concept inside transformer architecture. What is multi-head attention? Just think this way, we are processing a sentence and we want multiple experts to look at this sentence from different perspectives. Like one of the experts, head one, is focussing on syntax.

It is only understanding the grammatical structure and dependencies between the words. Another expert is employed to look at the semantics, meaning-based relationship between the concept, nothing to do with the grammar, the meaning. Grammatically, a frog is flying is correct, but semantics, when you look at it, it seems to be a little off the imagination that a frog is flying.



So basically, semantics focusses upon meaning-based relationship between concepts. Another head, another expert could be, it is focussing upon relative position and distance between the tokens. So basically, each of these head will have their own query, key and value vector, and they look at the sentence from a different perspective.

And then this idea about how the syntax is, how the semantics is, or position is, all these are embedded into a contextualised vector, which gets combined after all these experts or heads have processed the data. So it's like looking at the sentence from different perspective and then combining that knowledge for better understanding of the sentences, which adds a lot of value, you can think of it as. So that's your multi-head attention.

And why are we using it? Because it adds more context to the entire process. Now if you look at the, if you take a closer look at how the multi-head attention mechanism work, I explained to you, but here is the process. We begin with the project stage.

Instead of just using one version of query key and value, the model creates h different linear projections of query key and value. In other words, it makes multiple perspective of the same input data. Next in the process stage, each of these projections runs through its own attention function in parallel.

So instead of just one attention head, we now have multiple attention heads, each capturing different relationship in the data. Once all heads finish their work, we move to the concatenate step. Here the outputs from all attention heads are joined together into a single larger matrix.

This ensures we are bringing together the different perspective learned by each head. Finally, in the transform step, this concatenated output is passed through a final linear projection layer to produce the result. This step essentially blends all the different attention outputs into one unified representation that the model can use for downstream tasks.

So that is what the whole process is and mathematically we represent it as MHA of multi-head attention basically of QKV, concat, head 1, head 2 and whatever the weights are there. So in summary, multi-head attention lets the model look at the information from multiple angles at once, then combine those insights into a richer more powerful representation. To better understand the intuition behind multi-head attention, let's think of it as a team of analysts working on the same text passage.

Each analyst has a unique area of expertise and together they provide a complete picture. The first analyst is a grammar expert. This analyst focusses on the structural side tracking, subject verb relationships, sentence composition and syntactic patterns.

Essentially, they ensure that the model captures grammatical correctness and dependencies. The second analyst is a semantic specialist. Their role is to look beyond structure and focus on meaning.

They identify synonyms, related concepts and broader meaning-based relationships helping the model truly understand the context. The third analyst is a context coordinator. This one pays attention to how words relate to each other based on their positions in sequence.

For example, they notice which word comes before or after another and how that affects interpretation. By combining the insights of these different analysts, the model can simultaneously understand syntax, semantics and context. This is exactly what multi-head attention does.

Each head looks at the same input from a different perspective and when merged together the result is much richer and more comprehensive understanding of of language. So, putting it all together we start with the input embeddings which is your simple word embeddings, converting word into vector representation, projecting it onto Q , K and V vectors, then calculating the multi-attention weights and then combining the weights of



scores of each or basically the learnt representation of each head into a single representation and that contextualised output will be sent to the next layer to further process and finally give the output. So, that's how it works entirely.

That's what is your multi-head attention mechanism. So, what we did here is we went through. First, we went through the query key value mechanism.

These three components are the fundamental building blocks of self-attention. They allow models to selectively focus on relevant parts of the input rather than processing everything equally. Second, the multi-head architecture.

Instead of relying on a single attention mechanism, transformers use multiple heads in parallel. Each head looks at the data differently, some focussing on syntax, others on semantics, others on positional relationship. That's just an example.

That's how you can understand. By combining these perspectives, the model builds much richer representation than a single head approach ever could. And finally, the true transformer power this attention driven design enables transformers to handle long range dependencies, capture contextual meaning and interpret nuanced relationship in language. That is why transformers can understand not just individual words but how words interact across entire passages. Together, these concepts forms the backbone of today's most advanced models like your GPT, BERT and all that have completely revolutionised Natural Language Processing and extended far beyond into wised speech and multimodal AI. So that's the concept of Attention Mechanism.